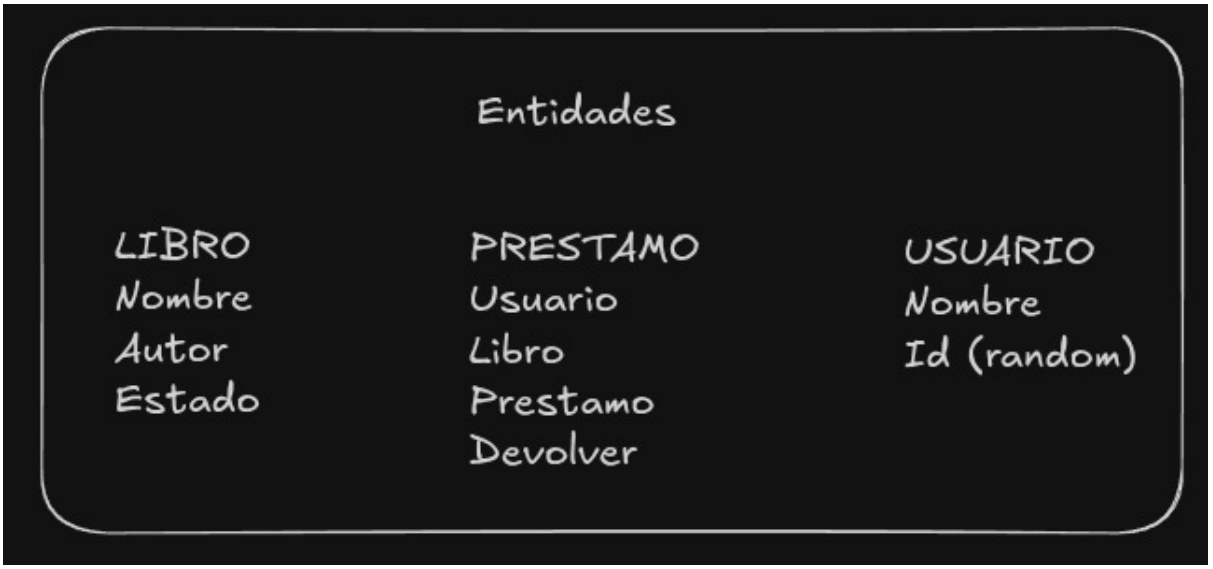
Universidad Nacional de la Defensa - UNDEF
CRUC - IUA
Ingeniería en Informática
Paradigmas de Programación II
Trabajo Práctico: Interacción de Objetos en POO

Estudiantes:
GARZÓN LOZA, Rocío Desiré
SANCHEZ SOLANO, Juan Cruz

1) Identificación de entidades

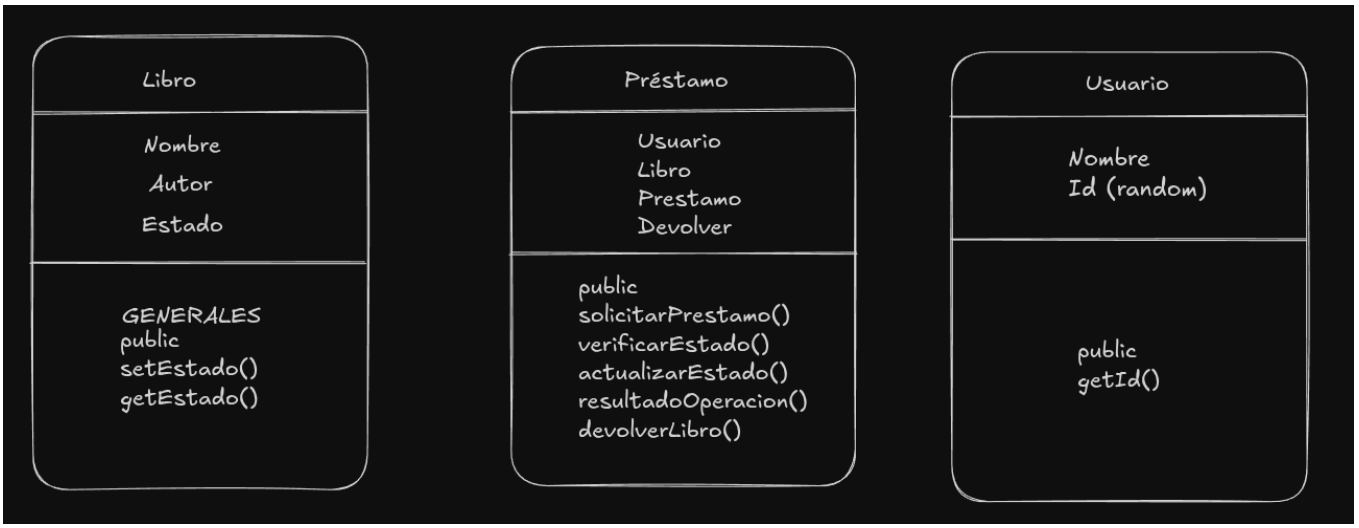
● Identificar las entidades principales del problema.
● Determinar qué responsabilidades tiene cada una.



En la imagen visualizamos las entidades con sus respectivos atributos a utilizar.
Libro: su responsabilidad es marcar su estado (disponible/prestado) e identificar su autor y título.
Usuario: la responsabilidad del usuario es proporcionar a préstamo su id.
Préstamo: su responsabilidad es procesar un préstamo o una devolución, cuyas operaciones son un resultado de utilizar “libro” y “usuario” combinados, es decir, préstamo nos permite relacionar estas entidades.
.

2) Diseño de clases. Para cada entidad identificada:

- Definir la clase correspondiente.
- Determina sus atributos y métodos.
- Justificar decisiones de diseño.



Decisiones del diseño:

Basándonos en la descripción de la operación, nombramos atributos que se amolden a las características del “libro” y decidimos hacer una clase libro la cual nos permite setear su estado de manera pública con setEstado() para poder ser utilizada en la interfaz por otras clases, y también nos permita retornar el valor de su atributo estado mediante getEstado() también de manera pública.

De la misma manera escogimos los atributos para el “usuario”, el cual decidimos que autogenere y retornará el valor de su id y nos permita visualizarlo públicamente.

Finalmente, decidimos utilizar la clase “préstamo” para relacionar “libro” y “usuario”. Es decir, los atributos de la clase préstamo serán del tipo “usuario” y “libro”, por otro lado, decidimos colocar dos banderas las cuales nos ayudaran en el desarrollo del código. Esta clase préstamo utilizará los métodos públicos de “libro” y “usuario”, con los cuales creamos métodos que nos permitan solicitar un préstamo, verificar el estado de un libro, actualizar ese estado, emitir un resultado de la operación y devolver el libro.

3) Constructores. Implementar en cada clase:

- Un constructor por defecto
- Un constructor sobrecargado con parámetros

Usuario:

```
public Usuario()
{
    // initialise instance variables
    id = 0;
    nombre = "";
}
public Usuario(String nombre)
{
    // initialise instance variables
    this.nombre = nombre;
    this.id = rand.nextInt(9000) + 1000;
}
```

Libro:

```
public
Libro()
{
    // initialise instance variables
    titulo = "";
    autor = "";
    estado = "";
}
Libro(String titulo, String autor, String estado)
{
    // initialise instance variables
    this.titulo = titulo;
    this.autor = autor;
    this.estado = estado;
}
```

Préstamo:

```
Prestamo()
{
    // initialise instance variables
    libro = new Libro("Caperucita", "Charles", "Disponible");
    usuario = new Usuario("Juan");
    prestamo = false;
    devolver = false;
}
Prestamo(Usuario usuario, Libro libro)
{
    // initialise instance variables
    this.libro = libro;
    this.usuario = usuario;
    prestamo = false;
    devolver = false;
}
```

4) Interacción de objetos

Implementar la funcionalidad necesaria para que:

- Un usuario pueda solicitar el préstamo de un libro
- Se verifique si el libro está disponible
- Se actualice el estado del libro
- Se informe el resultado de la operación
- Implementar la devolución del libro

Implementación:

```
private void actualizarEstado(){
    if(libro.getEstado().equals("Disponible")){
        libro.setEstado("Prestado");
    }else{
        libro.setEstado("Disponible");
    }
}

public

void solicitarPrestamo()
{
    if(verificarEstado()){
        actualizarEstado();
        prestamo = true;
    }else{
        prestamo = false;
    }
    resultadoOperacionPrestamo();
}

boolean verificarEstado()
{
    if(libro.getEstado().equals("Disponible")){
        return true;
    }else{
        return false;
    }
}

private void resultadoOperacionPrestamo(){
    if(prestamo){
        System.out.println("El prestamo se ha realizado con exito! id: " + usuario.getId());
        prestamo = false;
    }else{
        System.out.println("Libro no disponible");
    }

}
```

```
    private void resultadoOperacionDevolver(){
        if(devolver){
            System.out.println("La devolucion se ha realizado con exito! id: " + usuario.getId());
        }else{
            System.out.println("El libro no fue prestado");
        }
        devolver = false;
    }

    public void devolverLibro(){
        //prestamo = false;
        if(verificarEstado() == false){
        actualizarEstado();
        devolver = true;
    }else{
        devolver = false;
    }
        resultadoOperacionDevolver();
    }

}
```

5) Encapsulamiento

● Definir los atributos como privados
● Proporcionar los métodos de acceso necesarios
● Incluir al menos un método privado que colabore en la resolución del problema

Atributos:

Préstamo:

```
private
Usuario usuario;
Libro libro;
boolean prestamo;
boolean devolver;
```

LIbro:

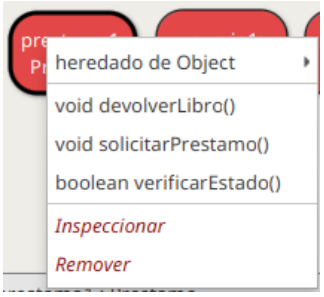
```
private
String titulo;
String autor;
String estado;
```

Usuario:

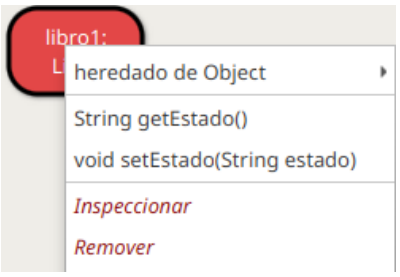
```
private
int id;
String nombre;
Random rand = new Random();
```

Acceso a métodos:
Los métodos con disponibilidad de acceso público se visualizan en las opciones del objeto, los que no se visualizan son de acceso privado:

Préstamo:



Libro:

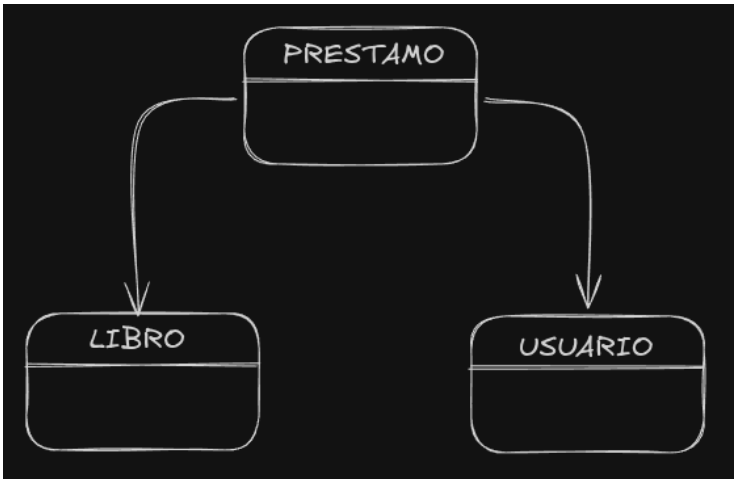


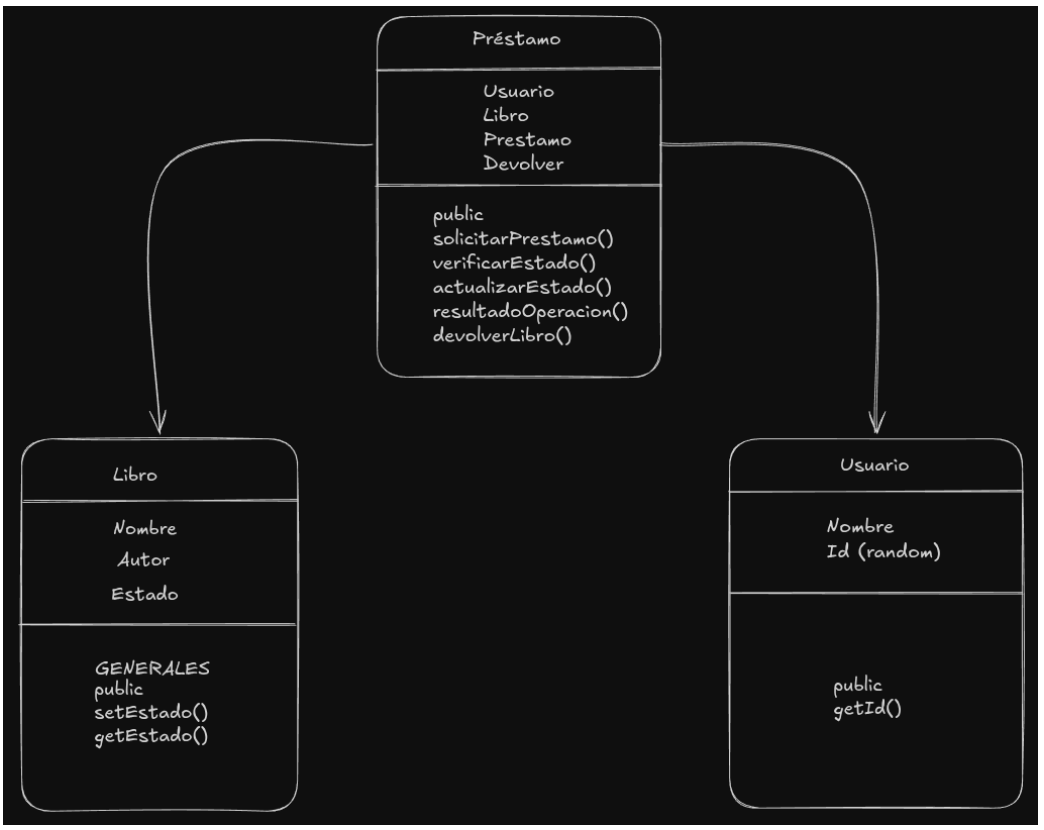
Usuario:

6) Diagrama de clases

Realizar el diagrama de clases correspondiente, donde se visualicen:

- Clases
- Atributos
- Métodos
- Relaciones entre clases

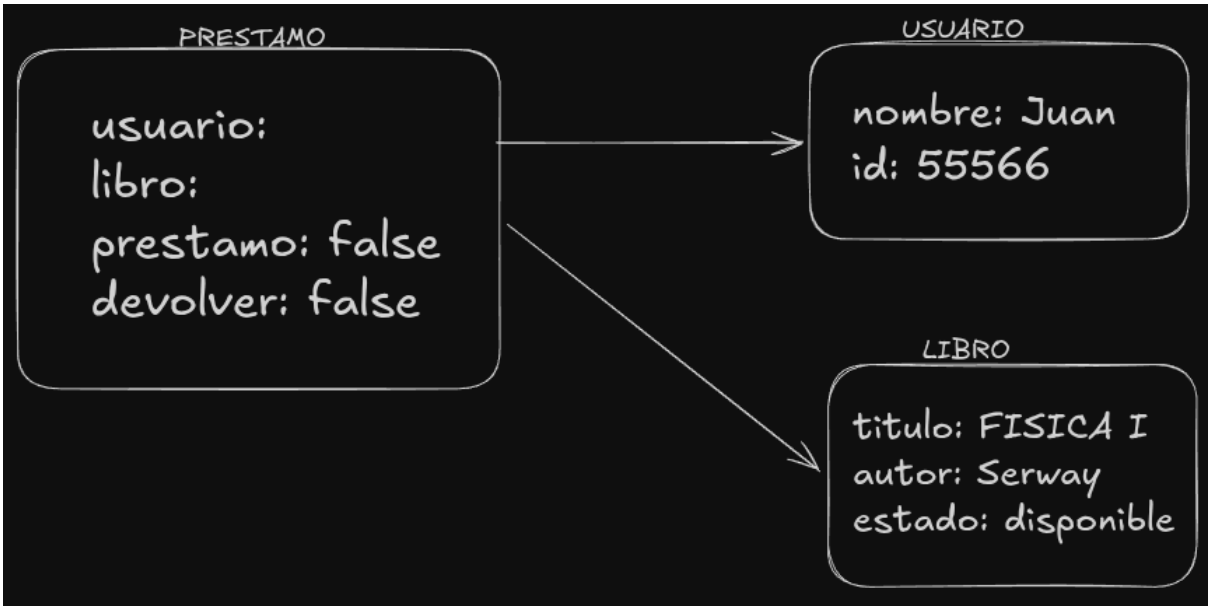




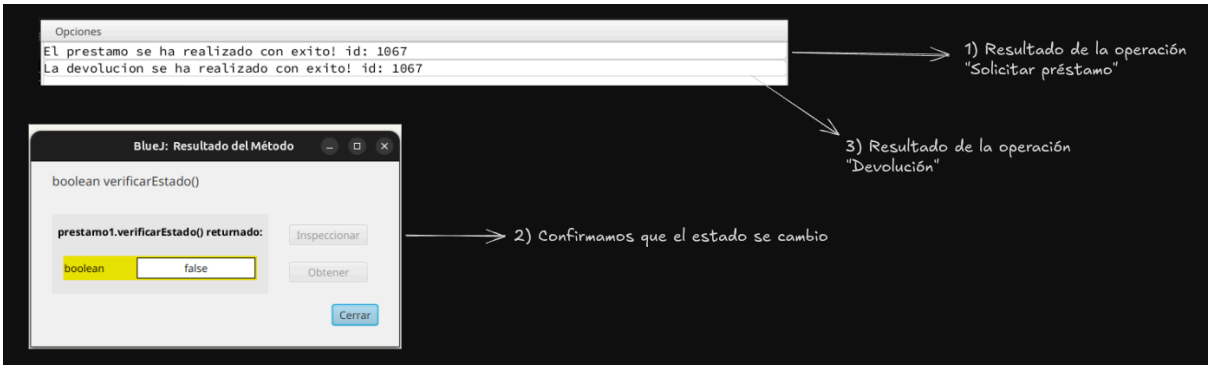
7) Diagrama de objetos

Representar un escenario de ejecución donde:

- Exista al menos un usuario
- Exista al menos un libro
- Se realice un préstamo



8) Prueba del sistema

Desde el entorno BlueJ:

- Crear instancias de las clases
- Ejecutar un caso de préstamo
- Ejecutar un caso de devolución